

רשתות קונבולוציה

אם ברשת fully connected רגילה אנחנו לומדים את הכל - כל המשקלים מכל איבר בקלט לכל צומת בשכבה הראשונה, ומכל צומת בשכבה הראשונה לשכבה השנייה, וכן הלאה עד הפלט - ברשת קונבולוציה אנחנו לומדים פילטרים - משקלים על סביבות של איברים בקלט (ובכל שכבה), כאשר הפילטר "משתכפל" (עם אותם משקלים) על פני כל הקלט. זה מאפשר לנו ללמוד הרבה פחות פרמטרים. זה מתאים לתמונות, כי שם המיקום של כל איבר בקלט הוא לא שרירותי אלא קשור לאיברים לידו. לא סתם שמים שכבות קונבולוציה אחת על השנייה. צריך¹ גם שכבות pooling שמצמצות את גודל השכבה הבאה:

- שכבת max pooling - לוקחת את המקסימום בכל אזור.
- שכבת average pooling.

הpooling הוא לוקאלי.

מקובל, בסוף רשת הקונבולוציה, לשטח את התמונה (מטריצה, אבל לא באמת כי לא עושים עליה כפל מטריצות) לווקטור - ולעשות שכבות fully connected כדי לבצע את הסיווג עצמו.

Batch Normalization

עושים בד"כ גם Batch Normalization. מחשבים לכל פיצ'ר ממוצע ושונות:

$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j} \quad \sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2$$

ומנרמלים את x :

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

Weight Decay

בשביל שהמשקלים יהיו קטנים, מוסיפים לloss:

$$L_{\text{new}}(W) = L_{\text{original}}(W) + \frac{\lambda}{2} \|W\|_2^2$$

זה מתבטא כשירודים בגרדיאנט:

$$W^t = \underbrace{(1 - \eta\lambda) W^{t-1}}_{\text{Weight Decay}} - \underbrace{\eta \left(\frac{\partial L_{\text{original}}}{\partial W} \right)}_{\text{Usual GD}}$$

¹בנוסף לשכבות האי-לינאריות, כמוכן

מה עושים כשהגאפ בין הtrain לtest גדול? (כלומר כשיש overfitting)

(אלו שאלות ממבחנים, שמדברים באופן כללי על רשתות - לא דווקא על קונבולוציה)

- להחליף בין הtrain לtest - לא רעיון טוב. הtest יותר קטן, וכנראה יהיה אפילו יותר overfitting.
- ליצור עוד תמונות באמצעות סיבובים - רעיון טוב.
- להשתמש ב-CNN במקום MLP - רעיון טוב. קונבולוציה מתאימה יותר לתמונות ומקטינה את מספר הפרמטרים באופן משמעותי.
- להוסיף skip connection - לא רעיון טוב. זה משפר את הtraining, אבל לא עוזר ל-overfitting.
- להוסיף weight decay - יעזור.
- להשתמש במודל יותר עמוק (עוד שכבות) - לא יעזור. זה רק יצור יותר overfitting.
- לעצור את האימון מוקדם יותר, בשלב שממזער את loss על הtest set - יעזור.
- להוסיף מומנטום - לא יעזור
- להשתמש ב-dropout - יעזור

Optimizers

שיטה שמגדירה איך לשנות פרמטרים כדי להשיג תוצאה יותר טובה נקראת *optimizer*.
תזכורת: *optimizer* שכבר למדנו זה SGD. המטרה למצוא $\theta = \arg \min \sum_{i=0}^N L(\theta, x, y)$
וה SGD מעדכן $\theta_i \leftarrow \theta_i - \eta \cdot \frac{\partial L}{\partial \theta_i}$.
יש לו כל מני בעיות - כמו הקושי בבחירת קצב הלמידה η , או היקלעות למינימומים מקומיים.
נרצה לראות עוד סוגי *optimizer*

מומנטום

משפר את SGD באמצעות ממוצע משוקלל מתגלגל של הגרדיאנטים הקודמים:

$$\nu_t = \gamma \nu_{t-1} + \eta \frac{\partial L}{\partial \theta_i} \quad \theta_i = \theta_i - \nu_t$$

(Adaptive Gradient Algorithm) AdaGrad

הרעיון: אם יש לנו פרמטרים גדולים, נרצה learning rate יותר קטן.
לכן בכל איטרציה:

$$c_{t,i} = \sum_{k=0}^t \left(\frac{\partial L}{\partial \theta_{k,i}} \right)^2 \quad \theta_{t+1,i} = \theta_{t,i} - \eta \frac{\frac{\partial L}{\partial \theta_{t,i}}}{\sqrt{c + \epsilon}}$$

$c_{t,i}$ בעצם מודד את ההיסטוריה של כל פרמטר, ולפי זה קובע כמה נעדכן אותו.
בעיה: c הופך להיות מאוד גדול, ואת העדכון מתאפס.
פתרון:

(Root Mean Square Propagation) RMSprop

כמו AdaGrad, אבל במקום לסכום על הכל עושים ממוצע משוקלל:

$$c_{t,i} = \gamma c_{t-1,i} + (1 - \gamma) \left(\frac{\partial L}{\partial \theta_{t,i}} \right)^2$$

(Adaptive Moment Estimation) Adam

משלב מומנטום וקצב למידה לכל פרמטר:

$$m_{t,i} = \gamma_1 m_{t-1,i} + (1 - \gamma_1) \left(\frac{\partial L}{\partial \theta_{t,i}} \right) \quad \nu_{t,i} = \gamma_2 \nu_{t-1,i} + (1 - \gamma_2) \left(\frac{\partial L}{\partial \theta_{t,i}} \right)^2$$
$$\theta_{t+1,i} = \theta_{t,i} - \eta \frac{m_{t,i}}{\sqrt{\nu_{t,i} + \epsilon}}$$

למידת מכונה
89-2511-05

מקליד: עידן אריה
מתרגל: ניר בן ארי
תאריך: 2026-06-01

AdamW

אותו דבר - אבל מוסיפים weight decay:

$$\theta_{t+1,i} = \theta_{t,i} - \eta \left(\frac{m_{t,i}}{\sqrt{\nu_{t,i} + \epsilon}} - \lambda \theta_i \right)$$